CmpE 160 Assignment 1
Report

Sude Naz Aslan – 2024400336
01
Spring 2026

1 Introduction

In this project, we made a shooter project that happens in space using Java and the StdDraw library, which enabled us to use images and make animations. The enemies, which are aliens, and our main character use their weapons to shoot and kill each other. Also, after killing an enemy, with changing chances, our character can catch extra hearts falling from above. Sample video of the game can be watched here. And bonus can be watched here.

2 Game Dynamics

2.1 Stage 1: Main Menu

On the main menu screen(Figure 1), which appears at the beginning of the game, there can be seen a huge title of the game and some other calibration parts. Under the title, there is a start button, FPS and speed values. Players can change those values by applying the instructions written below. Shortly, FPS is frames per second, and speed corresponds to the aircraft's speed. I used the StdDraw library's isKeyPressed method throughout the game to catch keyboard events. For adjusting FPS and speed, players should press the Q, D, A, and E buttons. FPS is important to adjust the game on different computers. I set it by playing with the pauseDuration variable. The character can also shoot and move on the main screen, which can be controlled by the arrow keys and the space character. When Enter is pressed, the game starts.

Figure 1: Main menu.

2.2 Stage 2: Gameplay

At the beginning of the game, players have 3 hearts and 0 score. Players aim to kill all enemies without losing all of their lives. If they lose all their hearts, which can be tracked from the top-right corner, the game ends. The player loses one life if an enemy shoots them.

The player can move and shoot by pressing specific buttons on the keyboard:

'Space': for shooting

'P': for pausing the game

'$\leftarrow \uparrow \rightarrow \downarrow$': for moving

By killing one enemy, the player acquires 30 points, which can be tracked from the score displayed at the top-left corner of the screen. There is a small explosion animation when the player kills an enemy. The animation is achieved by showing a bigger explosion effect right after the small-sized one.

Figure 2: Gameplay.

To track this, I initialized two arrays for explosions. One checks whether they are active, and the other is for timing. Each time I updated the status of an explosion, I increased the timer and checked whether it exceeded 10 ms. After that time interval, I drew a bigger explosion image.

Although there is an animation after each kill, the drop of extra hearts for the player is random, similar to enemy shooting. I followed the same strategy by creating a random number and checking whether it is less than 0.4 (as shown in Listing 1) Based on this condition, a heart may fall down for the player to catch, or only the animation may occur. The player can try to catch falling hearts to increase their chances of winning the game.

```
    for (int j = 0; j < enemy_number; j++) {
        if (!enemyActive[j]) continue;

        if (rectCollide(player_bullets_x[i], player_bullets_y[i], bullet_width, bullet_height,
                enemyX[j], enemyY[j], enemy_width, enemy_height)) {

            player_bullets_isactive[i] = false;
            enemyActive[j] = false;
            playerScore += 30;

            addExplosion(enemyX[j], enemyY[j]);

```

```
                // 40% chance to drop a heart
                if (Math.random() < 0.4) {
                    addHeart(enemyX[j], enemyY[j]);
                }
            }
        }
    }
```

Listing 1: Randomized heart drop

A new heart image is added next to the others at the top-right corner of the screen. When the player is shot by an enemy, the heart on the left-hand side disappears.

I updated each component appearing on the game screen by calling the necessary functions inside a while loop after clearing the canvas.

In terms of level difficulty, I increased the speed of enemy ships by decreasing the pauseDuration variable. Every 10 seconds, an if-condition decreases the duration until it becomes 25 to avoid crashing as can be seen on Listing 2)

```
        while (ingame) {
                timer++;
                if(timer%100==0){
                    pauseDuration=Math.max(25,pauseDuration-5);
                    System.out.println(pauseDuration);
                }
```

Listing 2: Difficulty adjustment.

The player can pause the game by pressing 'P', which freezes the game.(Figure 3) To continue, the same button should be pressed again.

Figure 3: Paused screen.

2.2.1 Enemies and Movement Mechanics

When the player presses Enter to start the game, the startGame() method is called and creates 8 enemies in total. I created several arrays to control enemies' speed, x-y position, shooting time interval, and life status. When the update function is called for enemies, I move them horizontally by their speeds. After checking whether they are alive or not, I increase their x coordinates in the enemyx array, which I use to draw enemies at that point. I allocated 1/7 of the canvas width to each enemy to move back and forth. By increasing the enemymove variable, I monitor whether an enemy is out of the space allocated for them. When this variable becomes bigger than canvasWidth/7, I multiply their speed by -1 and assign the enemymove variable to 0, as I did at the beginning.

```
    static void updateenemies(){
        enemy_move+=enemy_speed;
        //alloating space to each enemy to move between.
        if (enemy_move>canvas_width/7) {
            enemy_move = 0;
            for(int i=0;i<enemy_number;i++) {
                if (enemyActive[i]) {
                    enemy_speedd[i] *= -1;

                }
            }
```

Listing 3: Enemy movement

The shooting mechanism of the enemies depends on randomness and not strict rules. First, I initialized a list with 0s to monitor each enemy's shoot time. Each time I update enemies in the game, I increase the timer allocated to each enemy if the enemy is still alive. Each second in the game, by creating a shoot interval variable, I make a random variable generate a number between 0 and 1 for each alive enemy. After that, I check whether this number is less than 0.4 (Listing 4) in order to make the enemy shoot. If the generated number is greater than 0.4, the if-condition checks the next second in the game.

```
if (enemyShootTimer[i] > enemy_shoot_interval) {
                        if (Math.random() < 0.4) { // 40% chance to shoot
                            addEnemyBullet(enemyX[i], enemyY[i]);
                        }
                        enemyShootTimer[i] = 0;
                    }
```

Listing 4: Randomized enemy shooting.

2.2.2 Collision Detection

In order to check collisions, I first drew all the images with the values I gave and assigned them to a group of variables, including enemies, bullets, and the player. When the game starts, I check if there is a collision in each frame by calling the specific checkCollisions method with several parameters. To monitor enemy bullets and player bullets, I listed their x and y coordinates and their activeness. When the player or an enemy shoots, I add them and their values to the lists. Each time, I check collisions between player bullet–enemy, enemy bullet–player, and player–enemy by passing their coordinates and widths.

```
    static boolean rectCollide(double x1, double y1, double w1, double h1,
                               double x2, double y2, double w2, double h2) {

        //checking the collision conditions.
        return x1 - w1 / 2 < x2 + w2 / 2 &&
                x1 + w1 / 2 > x2 - w2 / 2 &&
                y1 - h1 / 2 < y2 + h2 / 2 &&
                y1 + h1 / 2 > y2 - h2 / 2;
    }
```

Listing 5: Collision check

Since collisions on both axes are required(Figure 4), the method compares the x and y conditions separately. We have two different rectangles, and the method first checks the x coordinates. It subtracts half of the width from the x value of the first rectangle (left edge) to compare it with the right edge of the second rectangle. Also, for a collision to occur, the x coordinate of the right edge of the first rectangle must be greater than the left edge coordinate of the second rectangle.

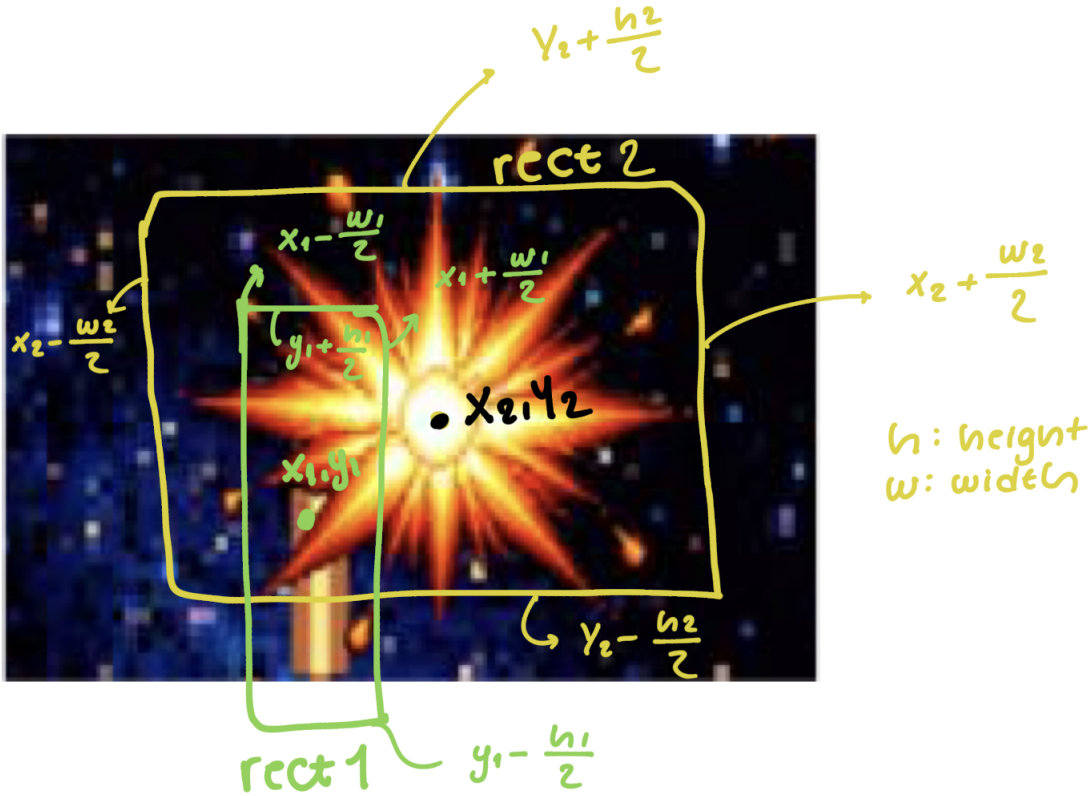

Figure 4: Visual collision representation.

The same strategy applies to the y coordinates as well. The bottom edge of the first rectangle must be below the top edge of the second rectangle. Meanwhile, its top edge should be above the bottom edge of the second rectangle.

2.3 Stage 3: End-Game

The game can end in two ways:

1.The player wins by killing all enemies. 2.The player loses by losing all of their hearts.

In the first scenario, a victory screen appears if the player manages to kill all of the enemies without losing their hearts.(figure 5) This is monitored by a function that checks the enemies' life status. On the victory screen, under the huge "VICTORY" title, the score, restart, and endgame options can be seen. The player can choose between restarting the game and terminating it by using the up and down buttons on the keyboard and pressing Enter afterwards. The selected option becomes >option<.

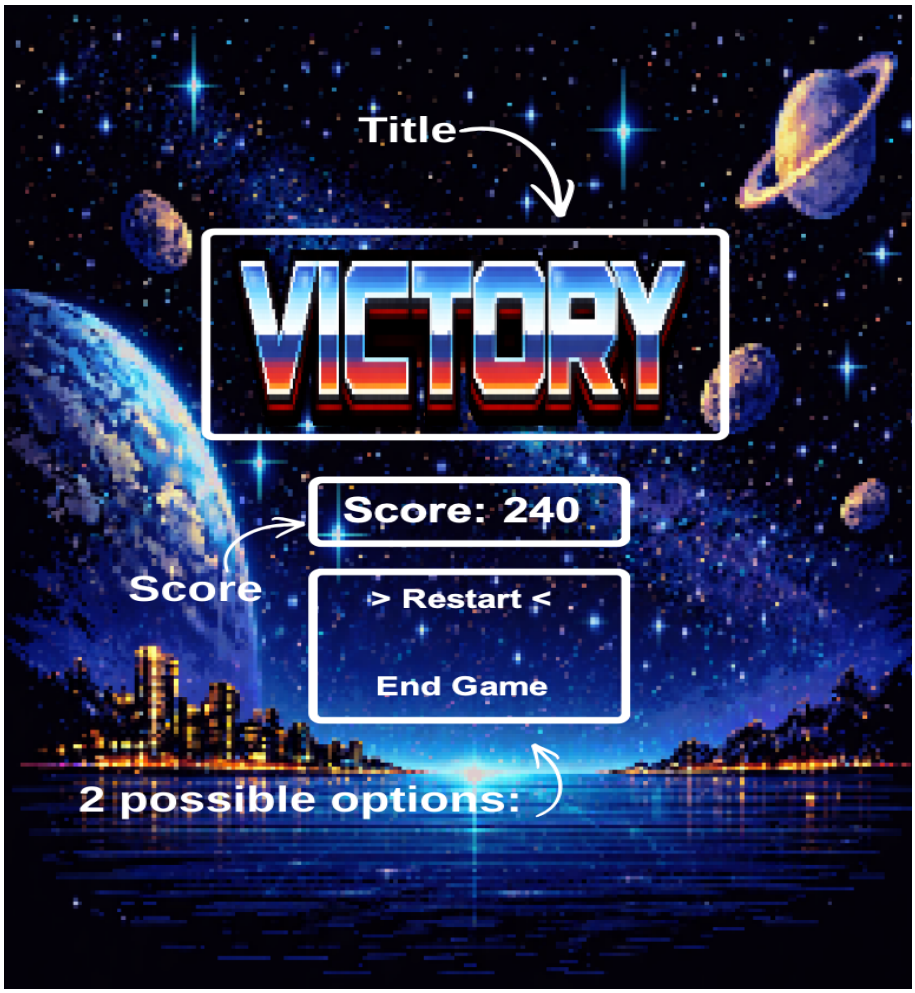

Figure 5: Endgame-victory.

The second scenario, failure, happens when the player loses all of their hearts. The screen is the same as the victory screen apart from the huge "GAME OVER" title at the center of the screen.

After pressing restart, the game starts with the previously chosen FPS and speed values. The endgame option closes the game window.

2.4 Bonus Part

Figure 6: Bonus part

In the bonus part , I made enemies move around the edges of the screen and randomly disappear. After they reappear, they start moving from the top-left side of the screen. I actually wrote random math functions for enemies including sin and cos functions to make unpredictable game. I also added sound and different explosion affect.